

ВВЕДЕНИЕ В СПИСКИ

Список представляет собой последовательность элементов, пронумерованных от 0, как символы в строке.

Создание списка

Чтобы создать список, нужно перечислить его элементы через запятую в квадратных скобках:

```
numbers = [2, 4, 6, 8, 10]
languages = ['Python', 'C#', 'C++', 'Java']
```

Список `numbers` состоит из 5 элементов, и каждый из них — целое число.

- `numbers[0] == 2;`
- `numbers[1] == 4;`
- `numbers[2] == 6;`
- `numbers[3] == 8;`
- `numbers[4] == 10.`

Список `languages` состоит из 4 элементов, каждый из которых — **строка**.

- `languages[0] == 'Python';`
- `languages[1] == 'C#';`
- `languages[2] == 'C++';`
- `languages[3] == 'Java'.`

Значения, заключенные в квадратные скобки и отделенные запятыми, называются **элементами списка**.

Список может содержать значения **разных типов данных**:

```
info = ['Timur', 1992, 61.5]
```

Список `info` содержит строковое значение, целое число и число с плавающей точкой.

- `info[0] == 'Timur';`
- `info[1] == 1992;`
- `info[2] == 61.5.`

Обычно элементы списка содержат данные одного типа, и на практике редко приходится создавать списки, содержащие элементы разных типов данных.

Пустой список

Создать пустой список можно двумя способами:

1. Использовать пустые квадратные скобки `[]` ;
2. Использовать встроенную функцию, которая называется `list` .

Следующие две строки кода создают пустой список:

```
mylist = [] # пустой список
mylist = list() # тоже пустой список
```

Вывод списка

Для вывода всего списка можно применить функцию `print()` :

```
numbers = [2, 4, 6, 8, 10]
languages = ['Python', 'C#', 'C++', 'Java']
print(numbers)
print(languages)
```

Функция `print()` выводит на экран элементы списка, в квадратных скобках, разделенные запятыми:

```
[2, 4, 6, 8, 10]
['Python', 'C#', 'C++', 'Java']
```

Обратите внимание, что вывод списка содержит квадратные скобки. Позже мы научимся выводить элементы списка в более удобном виде с помощью циклов или с помощью распаковки.

Встроенная функция list

Python имеет встроенную функцию `list()` , которая помимо создания пустого списка может преобразовывать некоторые типы объектов в списки.

Например, мы знаем, что функция `range()` создает последовательность целых чисел в заданном диапазоне. Для преобразования этой последовательности в список, мы пишем следующий код:

```
numbers = list(range(5))
```

Во время исполнения этого кода происходит следующее:

1. Вызывается функция `range()` , в которую в качестве аргумента передается число 5;
2. Эта функция возвращает последовательность чисел `0, 1, 2, 3, 4` ;
3. Последовательность чисел `0, 1, 2, 3, 4` передается в качестве аргумента в функцию `list()` ;
4. Функция `list()` возвращает список `[0, 1, 2, 3, 4]` ;
5. Список `[0, 1, 2, 3, 4]` присваивается переменной `numbers` .

Вот еще один пример:

```
even_numbers = list(range(0, 10, 2)) # список содержит четные числа 0, 2, 4, 6, 8
odd_numbers = list(range(1, 10, 2)) # список содержит нечетные числа 1, 3, 5, 7, 9
```

Точно так же с помощью функции `list()` мы можем создать список из символов строки. Для преобразования строки в список мы пишем следующий код:

```
s = 'abcde'
chars = list(s) # список содержит символы 'a', 'b', 'c', 'd', 'e'
```

Во время исполнения этого кода происходит следующее:

1. Вызывается функция `list()`, в которую в качестве аргумента передается строка `'abcde'`;
2. Функция `list()` возвращает список `['a', 'b', 'c', 'd', 'e']`;
3. Список `['a', 'b', 'c', 'd', 'e']` присваивается переменной `chars`.

основы работы со списками

Функция `len()`

Длиной списка называется количество его элементов. Для того чтобы посчитать длину списка, мы используем встроенную функцию `len()` (от слова length – длина).

Приведённый ниже код:

```
numbers = [2, 4, 6, 8, 10]
languages = ['Python', 'C#', 'C++', 'Java']

print(len(numbers))    # выводим длину списка numbers
print(len(languages))  # выводим длину списка languages

print(len(['apple', 'banana', 'cherry'])) # выводим длину списка,
состоящего из 3 элементов
```

ВЫВОДИТ:

```
5
4
3
```

Оператор принадлежности in

Оператор `in` позволяет проверить, содержит ли список некоторый элемент.

Рассмотрим следующий код:

```
numbers = [2, 4, 6, 8, 10]

if 2 in numbers:
    print('Список numbers содержит число 2')
else:
    print('Список numbers не содержит число 2')
```

Такой код проверяет, содержит ли список `numbers` число 2, и выводит соответствующий текст:

```
Список numbers содержит число 2
```

Мы можем использовать оператор `in` вместе с логическим оператором `not`. Например:

```
numbers = [2, 4, 6, 8, 10]

if 0 not in numbers:
    print('Список numbers не содержит нулей')
```

Индексация

При работе со строками мы использовали **индексацию**, то есть обращение к конкретному символу строки по его индексу. Аналогично, можно индексировать и списки.

Для индексации списков в Python используются квадратные скобки `[]`, в которых указывается индекс (номер) нужного элемента в списке:

Пусть `numbers = [2, 4, 6, 8, 10]`.

Таблица ниже показывает, как работает индексация:

Выражение	Результат	Пояснение
<code>numbers[0]</code>	2	первый элемент списка
<code>numbers[1]</code>	4	второй элемент списка
<code>numbers[2]</code>	6	третий элемент списка
<code>numbers[3]</code>	8	четвертый элемент списка
<code>numbers[4]</code>	10	пятый элемент списка

Как и в строках, попытка обратиться к элементу списка по несуществующему индексу:

```
print(numbers[17])
```

приводит к возникновению ошибки:

```
IndexError: index out of range
```

Срезы

Рассмотрим список `numbers = [2, 4, 6, 8, 10]`.

С помощью среза мы можем получить несколько элементов списка, создав диапазон индексов, разделенных двоеточием `numbers[x:y]`.

Приведённый ниже код:

```
print(numbers[1:3])
print(numbers[2:5])
```

ВЫВОДИТ:

```
[4, 6]
[6, 8, 10]
```

При построении среза `numbers[x:y]` первое число – это то место, где начинается срез (**включительно**), а второе – это место, где заканчивается срез (**невключительно**). Разрезая списки, мы создаем новые списки, по сути, подсписки исходного.

При использовании срезов со списками мы также можем опускать второй параметр в срезе `numbers[x:]` (но поставить двоеточие), тогда срез берется до конца списка. Аналогично если опустить первый параметр `numbers[:y]`, то можно взять срез от начала списка.

Срез `numbers[:]` возвращает копию исходного списка.

Как и в строках, мы можем использовать отрицательные индексы в срезах списков.

Использование срезов для изменения элементов в заданном диапазоне

Для изменения целого диапазона элементов списка можно использовать срезы.

Например, если мы хотим перевести на русский язык названия фруктов `'banana'`, `'cherry'`, `'kiwi'`, то это можно сделать с помощью среза.

Приведённый ниже код:

```
fruits = ['apple', 'apricot', 'banana', 'cherry', 'kiwi', 'lemon',  
          'mango']  
fruits[2:5] = ['банан', 'вишня', 'киви']  
  
print(fruits)
```

ВЫВОДИТ:

```
['apple', 'apricot', 'банан', 'вишня', 'киви', 'lemon', 'mango']
```

Операция конкатенации + и умножения на число *

Мы можем применять операторы `+` и `*` для списков подобно тому, как мы это делали со строками.

Приведённый ниже код:

```
print([1, 2, 3, 4] + [5, 6, 7, 8])
print([7, 8] * 3)
print([0] * 10)
```

ВЫВОДИТ:

```
[1, 2, 3, 4, 5, 6, 7, 8]
[7, 8, 7, 8, 7, 8]
[0, 0, 0, 0, 0, 0, 0, 0, 0, 0]
```

Для генерации списков, состоящих строго из повторяющихся элементов, умножение на число — самый короткий и правильный метод.

Мы также можем использовать расширенные операторы `+=` и `*=` при работе со списками.

Приведённый ниже код:

```
a = [1, 2, 3, 4]
b = [7, 8]
a += b    # добавляем к списку a список b
b *= 5    # повторяем список b 5 раз

print(a)
print(b)
```

ВЫВОДИТ:

```
[1, 2, 3, 4, 7, 8]
[7, 8, 7, 8, 7, 8, 7, 8, 7, 8]
```

Встроенные функции `sum()`, `min()`, `max()`

Встроенная функция `sum()` принимает в качестве параметра список чисел и вычисляет сумму его элементов.

Приведённый ниже код:

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
print('Сумма всех элементов списка =', sum(numbers))
```

ВЫВОДИТ:

```
Сумма всех элементов списка = 55
```

Встроенные функции `min()` и `max()` принимают в качестве параметра список и находят минимальный и максимальный элементы соответственно.

Приведённый ниже код:

```
numbers = [3, 4, 10, 3333, 12, -7, -5, 4]
print('Минимальный элемент =', min(numbers))
print('Максимальный элемент =', max(numbers))
```

ВЫВОДИТ:

```
Минимальный элемент = -7
Максимальный элемент = 3333
```

Отличие списков от строк

Несмотря на всю схожесть списков и строк, есть одно очень важное отличие: строки — **неизменяемые** объекты, а списки — **изменяемые**.

Приведённый ниже код:

```
s = 'abcdefg'
s[1] = 'x'      # пытаемся изменить 2 символ (по индексу 1) строки
```

приводит к возникновению ошибки:

```
object does not support item assignment
```


Приведённый ниже код:

```
numbers = [1, 2, 3, 4, 5, 6, 7]
numbers[1] = 101      # изменяем 2 элемент (по индексу 1) списка
print(numbers)
```

ВЫВОДИТ:

```
[1, 101, 3, 4, 5, 6, 7]
```

Запомни: изменять отдельные символы строк нельзя, однако можно изменять отдельные элементы списков. Для этого используем индексатор и оператор присваивания.

Добавление элементов

Метод `append()`

Для добавления нового элемента **в конец списка** используется метод `append()`.

Приведённый ниже код:

```
numbers = [1, 1, 2, 3, 5, 8, 13] # создаем список

numbers.append(21) # добавляем число 21 в конец списка
numbers.append(34) # добавляем число 34 в конец списка

print(numbers)
```

ВЫВОДИТ:

```
[1, 1, 2, 3, 5, 8, 13, 21, 34]
```

Обратите внимание, для того чтобы использовать метод `append()`, нужно, чтобы список был создан (при этом он может быть пустым).

Приведённый ниже код:

```
numbers = [] # создаем пустой список

numbers.append(1)
numbers.append(2)
numbers.append(3)

print(numbers)
```

ВЫВОДИТ:

```
[1, 2, 3]
```

Важно: мы не можем использовать индексы для установки значений элементов списка, если список пустой.

Приведённый ниже код:

```
numbers = [] # создаем пустой список

numbers[0] = 1
numbers[1] = 2
numbers[2] = 3

print(numbers)
```

приводит к возникновению ошибки:

```
IndexError: list assignment index out of range
```

Метод `extend()`

Можно также расширить список другим списком путём вызова метода `extend()`.

Приведённый ниже код:

```
numbers = [0, 2, 4, 6, 8, 10]
odds = [1, 3, 5, 7]

numbers.extend(odds)
print(numbers)
```

ВЫВОДИТ:

```
[0, 2, 4, 6, 8, 10, 1, 3, 5, 7]
```

Метод `extend()` как бы расширяет один список, добавляя к нему элементы другого списка.

Отличие между методами `append()` и `extend()` проявляется при добавлении строки к списку.

Приведённый ниже код:

```
words1 = ['iq option', 'stepik', 'beegreek']
words2 = ['iq option', 'stepik', 'beegreek']

words1.append('python')
words2.extend('python')

print(words1)
print(words2)
```

ВЫВОДИТ:

```
['iq option', 'stepik', 'beegreek', 'python']
['iq option', 'stepik', 'beegreek', 'p', 'y', 't', 'h', 'o', 'n']
```

Метод `append()` добавляет строку `'python'` целиком к списку, а метод `extend()` разбивает строку `'python'` на символы `'p'`, `'y'`, `'t'`, `'h'`, `'o'`, `'n'` и их добавляет в качестве элементов списка.

Удаление элементов

С помощью оператора `del` можно удалять элементы списка по определенному индексу.

Приведённый ниже код:

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9]
del numbers[5]    # удаляем элемент, имеющий индекс 5
print(numbers)
```

ВЫВОДИТ:

```
[1, 2, 3, 4, 5, 7, 8, 9]
```

Элемент под указанным индексом удаляется, а список перестраивается.

Обратите внимание на синтаксис удаления, так как он отличается от обычного вызова метода. При удалении элементов не надо передавать аргумент внутри круглых скобок.

Оператор `del` работает и со срезами: мы можем удалить целый диапазон элементов списка.

Приведённый ниже код:

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9]
del numbers[2:7]    # удаляем элементы с 2 по 6 включительно

print(numbers)
```

ВЫВОДИТ:

```
[1, 2, 8, 9]
```

Мы можем удалить все элементы на чётных позициях (0 , 2 , 4 , ...) исходного списка.

Приведённый ниже код:

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9]
del numbers[::2]

print(numbers)
```

ВЫВОДИТ:

```
[2, 4, 6, 8]
```

Вывод с помощью цикла for

Для вывода элементов списка **каждого на отдельной строке** можно использовать следующий код:

Вариант 1. Если нужны индексы элементов:

```
numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

for i in range(len(numbers)):
    print(numbers[i])
```

Мы передаем в функцию `range()` длину списка `len(numbers)`. В нашем случае длина списка `numbers` равна `11`. Таким образом, вызов функции `range(len(numbers))` имеет вид `range(11)`, и переменная цикла `i` последовательно перебирает все значения от `0` до `10`. Это означает, что выражение `numbers[i]` последовательно вернёт все элементы списка `numbers`. Такой способ итерации списка удобен, когда нам нужен не только сам элемент `numbers[i]`, но и его индекс `i`.

Вариант 2. Если индексы не нужны:

```
numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

for num in numbers:
    print(num)
```

Этот цикл пройдёт по списку `numbers`, придавая переменной цикла `num` значение каждого элемента списка (!) в отличие от предыдущего цикла, в котором переменная цикла «бегала» по индексам списка.

Если требуется выводить элементы списка на одной строке через пробел, то мы можем использовать необязательный параметр `end` функции `print()` :

```
numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

for num in numbers:
    print(num, end=' ')
```

Вывод с помощью распаковки списка

В Python есть удобный способ вывода элементов списка без использования цикла `for` .

Вариант 1. Вывод элементов списка через один символ пробела:

```
numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

print(*numbers)
```

Приведённый выше код выводит:

```
0 1 2 3 4 5 6 7 8 9 10
```

Вариант 2. Вывод элементов списка, каждого на отдельной строке:

```
numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

print(*numbers, sep='\n')
```

Приведённый выше код выводит:

```
0
1
2
3
4
5
6
7
8
9
10
```

Поскольку строки содержат символы, подобно тому, как списки содержат элементы, то мы можем использовать распаковку строк точно так же, как и распаковку списков.

Приведённый ниже код:

```
s = 'Python'

print(*s)
print()
print(*s, sep='\n')
```

ВЫВОДИТ

```
P y t h o n

P
y
t
h
o
n
```


Метод split()

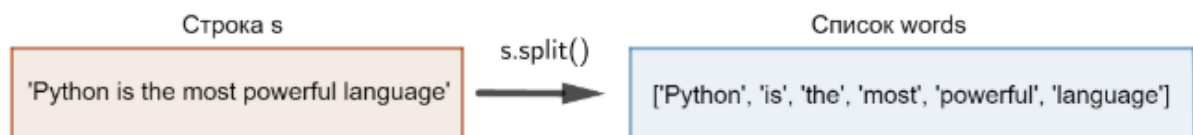
Метод `split()` разбивает строку на слова, используя в качестве разделителя последовательность пробельных символов, и возвращает список из этих слов.

Приведённый ниже код:

```
s = 'Python is the most powerful language'
words = s.split()
print(words)
```

ВЫВОДИТ:

```
['Python', 'is', 'the', 'most', 'powerful', 'language']
```



Таким образом, вызов метода `split()` разбивает **строку** на слова и возвращает **список**, содержащий все слова.

Рассмотрим следующий программный код:

```
numbers = input().split()
```

Если при запуске этой программы ввести строку `1 2 3 4 5`, то список `numbers` будет следующим `['1', '2', '3', '4', '5']`. Обратите внимание, что список будет состоять из строк (тип `str`), а не из чисел (тип `int`). Если требуется получить именно список чисел, то затем нужно элементы списка по одному преобразовать в числа с помощью команды `int()`.

Приведённый ниже код:

```
numbers = input().split()
for i in range(len(numbers)):
    numbers[i] = int(numbers[i])
print(numbers)
```

выводит (если на вход программе была подана строка `1 2 3 4 5`):

```
[1, 2, 3, 4, 5]
```

Необязательный параметр

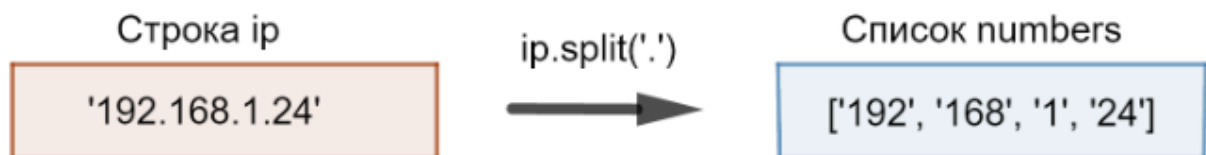
У метода `split()` есть необязательный параметр, который определяет, какой набор символов будет использоваться в качестве разделителя между элементами списка. Например, вызов метода `split('.')` вернёт список, полученный разделением исходной строки по символу `'.'`.

Приведённый ниже код:

```
ip = '192.168.1.24'
numbers = ip.split('.')    # указываем явно разделитель
print(numbers)
```

ВЫВОДИТ:

```
['192', '168', '1', '24']
```



Метод join()

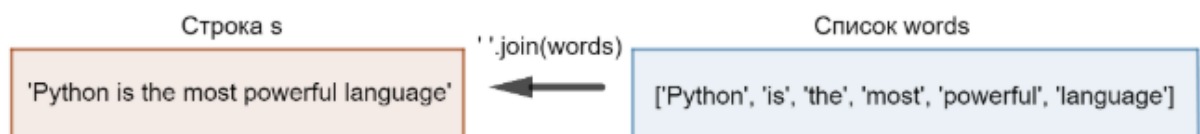
Метод `join()` собирает строку из элементов списка, используя в качестве разделителя строку, к которой применяется метод.

Приведённый ниже код:

```
words = ['Python', 'is', 'the', 'most', 'powerful', 'language']
s = ' '.join(words)
print(s)
```

ВЫВОДИТ:

```
Python is the most powerful language
```



Обратите внимание: все слова разделены одним пробелом, поскольку метод `join()` вызывался на строке, состоящей из одного символа пробела `' '`.

Рассмотрим ещё пару примеров:

```
words = ['Мы', 'учим', 'язык', 'Python']
print('*'.join(words))
print('-'.join(words))
print('?'.join(words))
print('!'.join(words))
print('*****'.join(words))
print('abc'.join(words))
print('123'.join(words))
```

Приведённый выше код выводит:

```
Мы*учим*язык*Python
Мы-учим-язык-Python
Мы?учим?язык?Python
Мы!учим!язык!Python
Мы*****учим*****язык*****Python
МыabcучимabcязыкabcPython
Мы123учим123язык123Python
```

Примечания

Примечание 1. Существует большая разница между результатами вызова методов `s.split()` и `s.split(' ')`. Разница в поведении проявляется, когда строка содержит несколько пробелов между словами.

Приведённый ниже код:

```
s = 'I love Python'
words1 = s.split()
words2 = s.split(' ')
print(words1)
print(words2)
```

ВЫВОДИТ:

```
['I', 'love', 'Python']
['I', 'love', '', 'Python']
```

Если в строке встречаются последовательные разделители, то они будут считаться как отдельные разделители, и между ними будет создана пустая строка (`''`).

Примечание 2. Методы `split()` и `join()` являются строковыми методами, они не могут применяться к спискам!

Приведённый ниже код:

```
print([1, 2].split())
```

приводит к возникновению ошибки:

```
AttributeError: 'list' object has no attribute 'split'
```

Примечание 3. Строковый метод `join()` не работает для списков, у которых элементы не строкового типа данных.

Приведённый ниже код:

```
numbers = [1, 2, 3, 4] # список чисел
s = '*'.join(numbers)
print(s)
```

приводит к возникновению ошибки:

```
TypeError: sequence item 0: expected str instance, int found
```

Примечание 4. На самом деле, метод `join()` в качестве аргумента может принимать не только список, но и строку.

Приведённый ниже код:

```
s = '+'.join('pygen')
print(s)
```

ВЫВОДИТ:

```
p+y+g+n
```

Тема урока: методы списков

1. Метод `insert()`
2. Метод `index()`
3. Метод `remove()`
4. Метод `pop()`
5. Метод `reverse()`
6. Метод `count()`
7. Метод `clear()`
8. Метод `copy()`
9. Метод `sort()`

Метод insert()

Метод `insert()` позволяет вставлять значение в список в заданной позиции. В него передается два аргумента:

1. `index` : индекс, задающий место вставки значения;
2. `value` : значение, которое требуется вставить.

Когда значение вставляется в список, список расширяется в размере, чтобы разместить новое значение. Значение, которое ранее находилось в заданной индексной позиции, и все элементы после него сдвигаются на одну позицию к концу списка.

Приведённый ниже код:

```
names = ['Gvido', 'Roman', 'Timur']
print(names)
names.insert(0, 'Anders')
print(names)
names.insert(3, 'Josef')
print(names)
```

ВЫВОДИТ:

```
['Gvido', 'Roman', 'Timur']
['Anders', 'Gvido', 'Roman', 'Timur']
['Anders', 'Gvido', 'Roman', 'Josef', 'Timur']
```

Если указан недопустимый индекс, то во время выполнения программы не происходит ошибки. Если задан индекс за пределами конца списка, то значение будет добавлено в конец списка. Если применен отрицательный индекс, который указывает на недопустимую позицию, то значение будет вставлено в начало списка.

Метод index()

Метод `index()` возвращает индекс первого элемента, значение которого равняется переданному в метод значению. Таким образом, в метод передается один параметр:

1. `value` : значение, индекс которого требуется найти.

Если элемент в списке не найден, то во время выполнения происходит ошибка.

Приведённый ниже код:

```
names = ['Gvido', 'Roman', 'Timur']
position = names.index('Timur')
print(position)
```

ВЫВОДИТ:

```
2
```

Приведённый ниже код:

```
names = ['Gvido', 'Roman', 'Timur']
position = names.index('Anders')
print(position)
```

приводит к возникновению ошибки:

```
ValueError: 'Anders' is not in list
```

Чтобы избежать таких ошибок, можно использовать метод `index()` вместе с оператором принадлежности `in` :

```
names = ['Gvido', 'Roman', 'Timur']
if 'Anders' in names:
    position = names.index('Anders')
    print(position)
else:
    print('Такого значения нет в списке')
```



Метод remove()

Метод `remove()` удаляет первый элемент, значение которого равняется переданному в метод значению. В метод передается один параметр:

1. `value` : значение, которое требуется удалить.

Метод уменьшает размер списка на один элемент. Все элементы после удаленного элемента смещаются на одну позицию к началу списка. Если элемент в списке не найден, то во время выполнения происходит ошибка.

Приведённый ниже код:

```
food = ['Рис', 'Курица', 'Рыба', 'Брокколи', 'Рис']  
print(food)  
food.remove('Рис')  
print(food)
```

выводит:

```
['Рис', 'Курица', 'Рыба', 'Брокколи', 'Рис']  
['Курица', 'Рыба', 'Брокколи', 'Рис']
```



Важно: метод `remove()` удаляет только первый элемент с указанным значением. Все последующие его вхождения остаются в списке. Чтобы удалить все вхождения, нужно использовать цикл `while` в связке с оператором принадлежности `in` и методом `remove`.

Метод pop()

Метод `pop()` удаляет элемент по указанному индексу и возвращает его. В метод `pop()` передается один **необязательный** аргумент:

1. `index` : индекс элемента, который требуется удалить.

Если индекс не указан, то метод удаляет и возвращает последний элемент списка. Если список пуст или указан индекс за пределами диапазона, то во время выполнения происходит ошибка.

Приведённый ниже код:

```
names = ['Gvido', 'Roman', 'Timur']
item = names.pop(1)
print(item)
print(names)
```

ВЫВОДИТ:

```
Roman
['Gvido', 'Timur']
```

Метод count()

Метод `count()` возвращает количество элементов в списке, значения которых равны переданному в метод значению.

Таким образом, в метод передается один параметр:

1. `value` : значение, количество элементов, равных которому, нужно посчитать.

Если значение в списке не найдено, то метод возвращает 0.

Приведённый ниже код:

```
names = ['Timur', 'Gvido', 'Roman', 'Timur', 'Anders', 'Timur']
cnt1 = names.count('Timur')
cnt2 = names.count('Gvido')
cnt3 = names.count('Josef')

print(cnt1)
print(cnt2)
print(cnt3)
```

ВЫВОДИТ:

```
3
1
0
```

Метод reverse()

Метод `reverse()` инвертирует порядок следования значений в списке, то есть меняет его на противоположный.

Приведённый ниже код:

```
names = ['Gvido', 'Roman', 'Timur']
names.reverse()
print(names)
```

ВЫВОДИТ:

```
['Timur', 'Roman', 'Gvido']
```



Существует большая разница между вызовом метода `names.reverse()` и использованием среза `names[::-1]`. Метод `reverse()` меняет порядок элементов на обратный **в текущем списке**, а срез создаёт копию списка, в котором элементы следуют в обратном порядке.

Метод clear()

Метод `clear()` удаляет все элементы из списка.

Приведённый ниже код:

```
names = ['Gvido', 'Roman', 'Timur']
names.clear()
print(names)
```

ВЫВОДИТ:

```
[]
```

Метод `copy()`

Метод `copy()` создаёт поверхностную копию списка.

Приведённый ниже код:

```
names = ['Gvido', 'Roman', 'Timur']
names_copy = names.copy()           # создаем поверхностную копию
                                     # списка names

print(names)
print(names_copy)
```

ВЫВОДИТ:

```
['Gvido', 'Roman', 'Timur']
['Gvido', 'Roman', 'Timur']
```

Аналогичного результата можно достичь с помощью срезов или функции `list()` :

```
names = ['Gvido', 'Roman', 'Timur']
names_copy1 = list(names)           # создаем поверхностную копию с
помощью функции list()
names_copy2 = names[:]              # создаем поверхностную копию с
помощью среза от начала до конца
```

Метод sort()

В Python списки имеют встроенный метод `sort()`, который сортирует элементы списка по возрастанию или убыванию.

Приведённый ниже код:

```
a = [1, 7, -3, 9, 0, -67, 34, 12, 45, 1000, 6, 8, -2, 99]
a.sort()
print('Отсортированный список:', a)
```

выводит:

```
Отсортированный список: [-67, -3, -2, 0, 1, 6, 7, 8, 9, 12, 34, 45, 99, 1000]
```

По умолчанию метод `sort()` сортирует список по возрастанию. Если требуется отсортировать список по убыванию, необходимо явно указать параметр `reverse = True`.

Приведённый ниже код:

```
a = [1, 7, -3, 9, 0, -67, 34, 12, 45, 1000, 6, 8, -2, 99]
a.sort(reverse=True) # сортируем по убыванию
print('Отсортированный список:', a)
```

выводит:

```
Отсортированный список: [1000, 99, 45, 34, 12, 9, 8, 7, 6, 1, 0, -2, -3, -67]
```

Тема урока: списочные выражения

Создание списков

Для того чтобы создать список, состоящий из 10 нулей, мы можем использовать приведённый ниже код:

```
zeros = []  
for i in range(10):  
    zeros.append(0)
```

В Python, однако, есть более простой и компактный способ для создания такого списка. Мы можем использовать оператор умножения списка на число:

```
zeros = [0] * 10
```

Для создания списков, заполненных по более сложным правилам, нам приходится явно использовать цикл `for`.

Например, для создания списка целых чисел от 0 до 9 мы вынуждены писать такой код:

```
numbers = []  
for i in range(10):  
    numbers.append(i)
```

Приведённый выше код хоть и не является сложным, однако достаточно громоздок.

Списочные выражения

В Python есть механизм для создания списков из неповторяющихся элементов. Такой механизм называется **списочное выражение** (list comprehension).

Предыдущий код можно записать следующим образом:

```
numbers = [i for i in range(10)]
```

Общий вид списочного выражения следующий:

```
[выражение for переменная in последовательность]
```

где **переменная** – имя некоторой переменной, **последовательность** – последовательность значений, которые она принимает (список, строка или объект, полученный при помощи функции `range`), **выражение** – некоторое выражение, как правило, зависящее от использованной в списочном выражении переменной, которым будут заполнены элементы списка.

Примеры использования списочных выражений

1. Создать список, заполненный 10 нулями можно и при помощи списочного выражения:

```
zeros = [0 for i in range(10)]
```

2. Создать список, заполненный квадратами целых чисел от 0 до 9 можно так:

```
squares = [i ** 2 for i in range(10)]
```

3. Создать список, заполненный кубами целых чисел от 10 до 20 можно так:

```
cubes = [i ** 3 for i in range(10, 21)]
```

4. Создать список, заполненный символами строки:

```
chars = [c for c in 'abcdefg']  
print(chars)
```

Считывание входных данных

При решении многих задач из предыдущих уроков мы считывали начальные данные (строки, числа) и заполняли ими список. С помощью списочных выражений процесс заполнения списка можно заметно сократить.

Например, если сначала вводится число `n` – количество строк, а затем на вход поступают сами строки, то создать список можно так:

```
n = int(input())  
lines = [input() for _ in range(n)]
```

Можно опустить описание переменной `n` :

```
lines = [input() for _ in range(int(input()))]
```

Если требуется считать список чисел, то необходимо добавить преобразование типов:

```
numbers = [int(input()) for _ in range(int(input()))]
```

Обратите внимание, мы используем символ `_` в качестве имени переменной цикла, поскольку она не используется.

Условия в списочном выражении

В списочных выражениях можно использовать условный оператор. Например, если требуется создать список чётных чисел от 0 до 20, то мы можем написать такой код:

```
evens = [i for i in range(21) if i % 2 == 0]
```


Вложенные циклы

В списочном выражении можно использовать вложенные циклы.

Приведённый ниже код:

```
numbers = [i * j for i in range(1, 5) for j in range(2)]  
print(numbers)
```

ВЫВОДИТ:

```
[0, 1, 0, 2, 0, 3, 0, 4]
```

Приведённый выше код код равнозначен следующему:

```
numbers = []  
  
for i in range(1, 5):  
    for j in range(2):  
        numbers.append(i * j)  
print(numbers)
```

Подводя итог

Пусть `word = 'Hello'`, `numbers = [1, 14, 5, 9, 12]`, `words = ['one', 'two', 'three', 'four', 'five', 'six']`.

Списочное выражение	Результирующий список
<code>[0 for i in range(10)]</code>	<code>[0, 0, 0, 0, 0, 0, 0, 0, 0, 0]</code>
<code>[i ** 2 for i in range(1, 8)]</code>	<code>[1, 4, 9, 16, 25, 36, 49]</code>
<code>[i * 10 for i in numbers]</code>	<code>[10, 140, 50, 90, 120]</code>
<code>[c * 2 for c in word]</code>	<code>['HH', 'ee', 'll', 'll', 'oo']</code>
<code>[m[0] for m in words]</code>	<code>['o', 't', 't', 'f', 'f', 's']</code>
<code>[i for i in numbers if i < 10]</code>	<code>[1, 5, 9]</code>
<code>[m[0] for m in words if len(m) == 3]</code>	<code>['o', 't', 's']</code>